Task 3

Heuristic Graph Pathfinding Agent Search Engine

AI Internship Report | Name: FIZA ASGHAR | Date: 13 OCTOBER 2026

1. Objective

To code an optimization search framework that lets a virtual agent navigate complex grid mazes efficiently. The task requires heuristic pathfinding algorithms (A* Search, Dijkstra's algorithm, or Q-Learning) written from scratch, tests across grids with varying obstacle layouts, tracking of metrics such as step count, runtime and expanded-node density, and export of visualization logs that show path optimization.

2. Summary of Results

A* Search and Dijkstra's algorithm were implemented from scratch in Python and compared on 300 runs over 150 randomly generated grids and mazes. Both algorithms always returned paths of identical, optimal cost, but A* needed far less exploration.

Metric (mean over 100 solved grids)	Dijkstra	A*	A* vs Dijkstra
Expanded nodes	1259.7	334.4	73.5% fewer
Expanded-node density	94.9%	29.4%	65.5 points lower
Runtime	3.71 ms	1.11 ms	about 3.3x faster
Peak frontier size (memory)	31.9	97.9	about 3x more
Path cost / steps	optimal	optimal	identical

3. Problem Setup

- The environment is a 2D grid. Each cell is either free (0) or an obstacle (1). The agent starts in the top-left area and must reach the goal in the opposite corner.
- Movement is 4-directional (up, down, left, right) with a cost of 1 per step. An optional 8-directional mode is also implemented, where diagonal moves cost the square root of 2 and may not cut across the corner of a wall.
- The path cost is the sum of step costs; an algorithm is optimal if it returns the lowest possible cost.

4. Algorithms

4.1 Dijkstra's algorithm

Dijkstra repeatedly expands the cell with the lowest cost-so-far g(n). It has no knowledge of where the goal is, so it spreads outward in all directions like a ripple and is guaranteed to find the cheapest path.

4.2 A* Search

A* expands the cell with the lowest f(n) = g(n) + h(n), where h(n) is a heuristic estimate of the remaining distance to the goal. Here h is the **Manhattan distance** (|row difference| + |column difference|) for 4-direction movement and the octile distance for 8-direction movement. Both heuristics are admissible (they never overestimate the true remaining cost), so A* is still guaranteed to return an optimal path, while being steered toward the goal.

4.3 Implementation details

- A single search function implements both algorithms; Dijkstra is simply A* with h(n) = 0.
- The open list is a binary-heap priority queue (Python's heapq). Stale heap entries are skipped (lazy deletion), and ties are broken in favour of the smaller heuristic value.
- The path is rebuilt by following parent pointers from the goal back to the start.
- An assertion inside the experiment loop checks that A* and Dijkstra always agree on whether a path exists and on its cost, which verifies optimality on every test case.

5. Experimental Design

- **Random grids:** 3 sizes (20x20, 40x40, 60x60) x 4 obstacle densities (10%, 20%, 30%, 40%) x 10 random seeds = 120 grids.
- **Mazes:** 3 sizes (21x21, 41x41, 61x61) x 10 seeds = 30 mazes, generated with a recursive-backtracker algorithm. About 8% of inner walls are then removed so that several routes exist; a perfect maze has only one path and cannot show the benefit of a heuristic.
- Every grid is solved by both algorithms, giving 300 runs in total. Grids with no possible path are detected and excluded from the averages.

Metrics recorded for every run:

Metric	Meaning
Steps / cost	Number of moves and total cost of the returned path
Expanded nodes	Cells taken from the open list and processed (search effort)
Expanded-node density	Expanded nodes divided by the number of free cells, in percent
Peak frontier size	Largest size of the open list (a measure of memory use)
Runtime	Wall-clock time of the search in milliseconds

6. Results

6.1 Effect of obstacle density (random grids)

Obstacle density	Solvable grids (of 30)	Dijkstra expanded	A* expanded	Dijkstra density	A* density
10%	28	1635.5	163.6	100.0%	11.1%
20%	27	1490.7	200.9	99.6%	14.5%
30%	14	859.7	311.0	92.3%	31.9%
40%	1	163.0	124.0	68.5%	52.1%

Note: at 40% obstacles only one grid out of 30 had any path, so that row is not statistically meaningful.

6.2 Effect of grid size (random grids)

Grid size	Dijkstra expanded	A* expanded	Dijkstra density	A* density
20 x 20	310.9	64.2	95.5%	21.0%
40 x 40	1306.1	191.3	99.3%	15.0%
60 x 60	2990.4	417.4	99.4%	14.6%

6.3 Mazes

Maze size	Path steps	Dijkstra expanded	A* expanded	Dijkstra density	A* density
21 x 21	53.4	159.4	108.8	76.3%	52.1%
41 x 41	106.0	756.6	422.9	89.8%	50.2%
61 x 61	165.6	1856.9	1363.8	97.7%	71.7%

6.4 Detailed demonstration cases

Case	Algorithm	Steps	Expanded	Density	Time (ms)
Random 40x40, 15% obstacles	Dijkstra	78	1345	99.7%	3.58
	A*	78	86	6.4%	0.62
Random 40x40, 30% obstacles	Dijkstra	82	1111	98.3%	4.12
	A*	82	429	38.0%	2.04
Braided maze 41x41	Dijkstra	118	736	87.4%	2.62
	A*	118	367	43.6%	1.71

6.5 Visualization logs

For each demonstration case both algorithms are drawn side by side. Light-blue cells were expanded, dark cells are obstacles, the red line is the final path, the green circle is the start and the star is the goal.

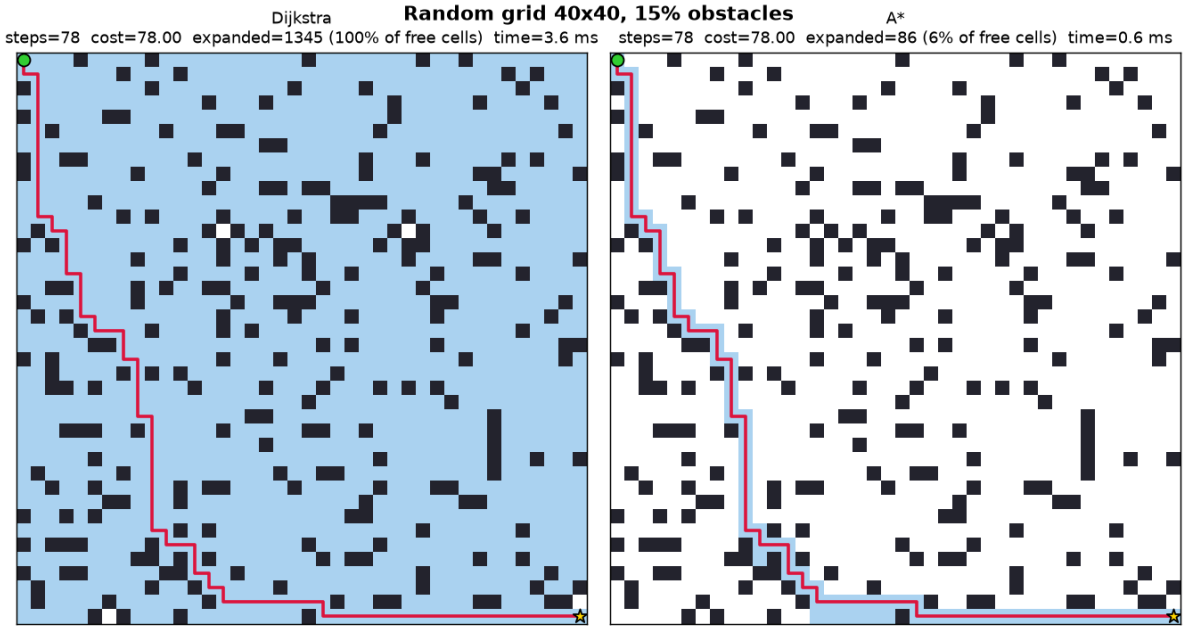


Figure 1: Dijkstra vs A on a 40x40 grid with 15% obstacles.*

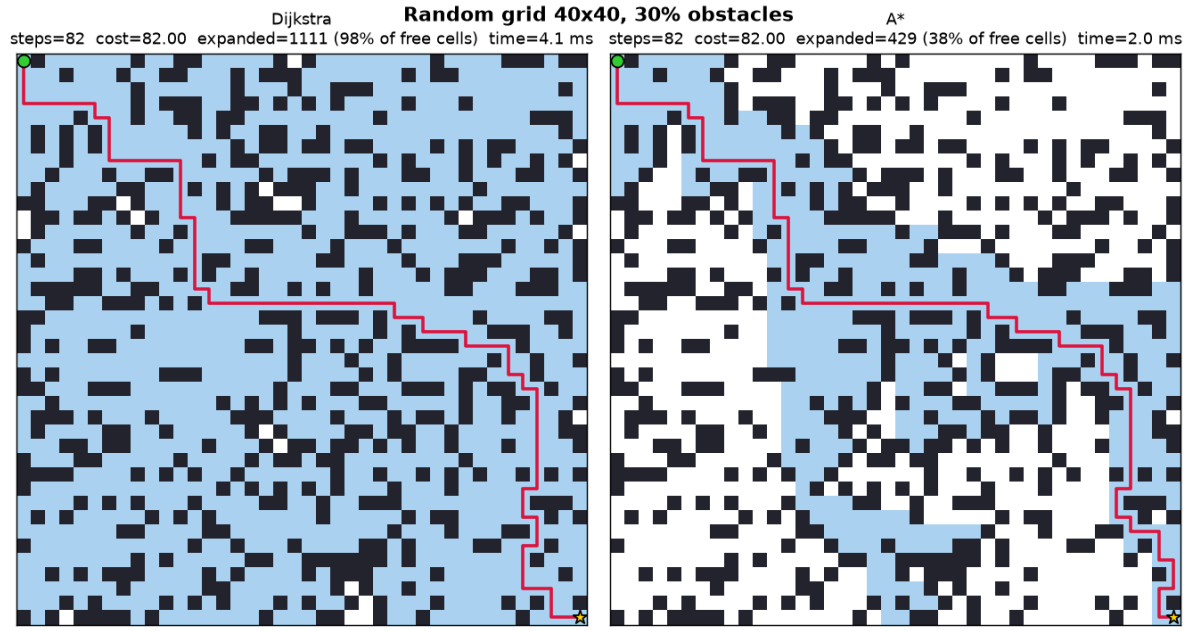


Figure 2: Dijkstra vs A on a 40x40 grid with 30% obstacles.*

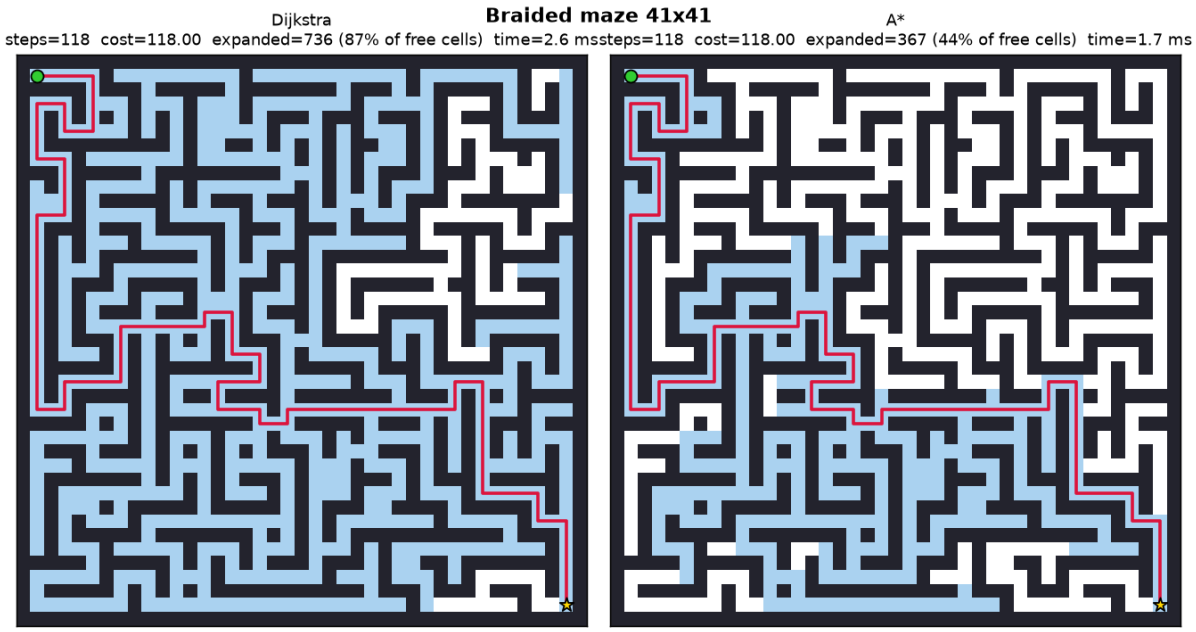


Figure 3: Dijkstra vs A on a 41x41 braided maze.*

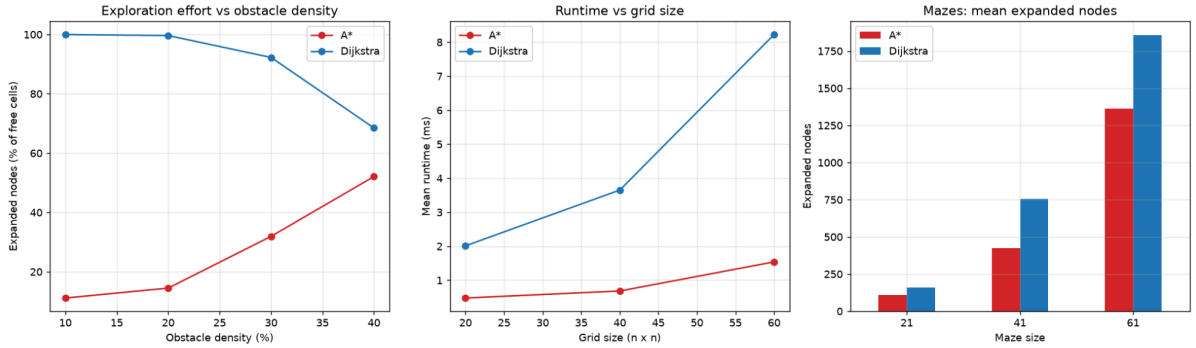


Figure 4: Exploration effort vs obstacle density, runtime vs grid size, and expanded nodes in mazes.

In addition, an animated GIF (search_animation_maze.gif) replays the order in which each algorithm explores the maze, and JSON logs store the grid, the exact path, the expansion order and all metrics of each demonstration run.

7. Analysis and Discussion

- **Same quality, much less work.** In all 100 solved grids A* returned exactly the same path cost as Dijkstra while expanding fewer or equal nodes. Per grid, the reduction in expanded nodes ranged from 2.5% to 96.3% (median 82.2%).
- **Why A* is faster.** Dijkstra explores in every direction and in open grids ends up examining almost every free cell (99 to 100%). A*'s heuristic points it toward the goal, so in open grids it explored only 6% to 15% of the free cells.
- **The advantage grows with grid size.** Dijkstra's effort grows with the area of the grid (2990 expanded nodes at 60x60), while A* stayed near 15% of the free cells, so it expanded about 7 times fewer nodes at 60x60 than Dijkstra.
- **The advantage shrinks as obstacles increase.** With many walls the straight-line guess is less accurate, because the real path must detour around obstacles. The same

effect appears in mazes, where A* still helped (about 27% to 44% fewer expanded nodes) but less than on open grids.

- **Memory trade-off.** A* keeps a larger open list (peak frontier about 98 vs 32 on average), because it pushes deep toward the goal while remembering many alternatives. A* therefore trades some memory for a large saving in time and exploration.
- **Runtime.** A* was about 3.3 times faster on average (1.11 ms vs 3.71 ms), which is lower than the 73.5% reduction in expanded nodes would suggest because each A* expansion also costs a heuristic computation.
- **Unsolvable grids.** Without any path, both algorithms must exhaust the reachable region and show the same cost. 50 of the 150 grids had no path, almost all at 30% and 40% obstacles. For 4-direction movement a connected route across a large random grid needs roughly 59% free cells (the site-percolation threshold), so grids near 40% obstacles are often blocked.

8. Limitations

- The priority queue uses Python's built-in heapq module; the search algorithms themselves are written from scratch, but the heap data structure is not.
- Only one heuristic (Manhattan, or octile in diagonal mode) was tested. Comparing several heuristics, or weighted A*, was not part of this study.
- At high obstacle densities few grids were solvable (14 at 30%, 1 at 40%), so conclusions for those settings are weak; a fixed number of solvable grids per setting would be better.
- Runtime was measured in Python on a single machine for small grids (up to 61x61) and varies from run to run; the expanded-node counts are deterministic and are the more reliable comparison.
- Random obstacle grids are not realistic maps. Q-Learning, dynamic obstacles and moving agents were not explored.

9. Conclusion

A* Search and Dijkstra's algorithm were implemented from scratch and tested on 300 runs over random grids and mazes of different sizes and densities. Both always found the optimal path, but A* expanded 73.5% fewer nodes and ran about 3.3 times faster on average, at the cost of a larger frontier. Its advantage is largest on big, open grids and smaller in dense or maze-like environments. Future work could add Q-Learning for comparison, more heuristics, weighted or bidirectional search, and dynamic obstacles.

10. Deliverables

- pathfinding_agent.py (complete source code: grid generation, A*, Dijkstra, experiments, plotting and logging)
- results/experiments.csv (all 300 runs) and results/summary.csv (averages per setting)
- results/figures/ (path_random_open.png, path_random_dense.png, path_maze.png, summary_comparison.png, search_animation_maze.gif)
- results/logs/ (random_open.json, random_dense.json, maze.json)